

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
“ЛЭТИ” ИМ.В.И.УЛЬЯНОВА (ЛЕНИНА)»
КАФЕДРА МОЭВМ**

**ОТЧЕТ
по лабораторно-практической работе № 10
«Протоколирование работы приложения»
по дисциплине «Объектно - ориентированное программирование на
языке Java»**

Выполнил: Шарапов И.Д.

Факультет: КТИ

Группа: №3312

Подпись преподавателя: _____

Санкт-Петербург

2024

Содержание

Цель работы	3
Перечень используемых типов сообщений	3
Конфигурационный файл	3
Текст программы	5
Приложение	16

Цель работы

Знакомство с методами протоколирования работы приложения с использованием библиотеки Log4j.

Перечень используемых типов сообщений

Уровень	Назначение	Использование
<i>DEBUG</i>	Диагностика и отладка	Детали выполнения операций, состояния объектов.
<i>INFO</i>	Подтверждение успешного выполнения	Начало работы, успешное завершение операций.
<i>WARN</i>	Предупреждение о проблемах	Некорректное поведение пользователя, отмена операций.
<i>ERROR</i>	Сообщения об ошибках	Исключения и сбои при выполнении операций.

Таблица 1 – Типы сообщений в лог-файлах

Конфигурационный файл

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %c{1}:%L - %msg%n"/>
    </Console>
    <File name="File" fileName="logs/app.log" append="true">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %c{1}:%L - %msg%n"/>
    </File>
  </Appenders>
  <Loggers>
    <Root level="debug">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="File"/>
    </Root>
  </Loggers>
</Configuration>
```

Рисунок 1 – Содержимое конфига логов
Лог-файлы

```

2024-12-04 14:17:15 [main] INFO Main:553 - Запуск приложения GAI System.
2024-12-04 14:17:16 [main] INFO Main:557 - Приложение успешно запущено.
2024-12-04 14:17:17 [AWT-EventQueue-0] INFO Main:206 - Пользователь нажал кнопку 'Загрузить'.
2024-12-04 14:17:17 [AWT-EventQueue-0] INFO Main:210 - Поток для загрузки данных запущен.
2024-12-04 14:17:17 [Thread-0] INFO Main:389 - Начата загрузка данных из XML файла.
2024-12-04 14:17:20 [Thread-0] INFO Main:398 - Выбран файл для загрузки: D:\Projects\00P_ETU\input.xml
2024-12-04 14:17:21 [Thread-0] INFO Main:426 - Данные успешно загружены из файла: D:\Projects\00P_ETU\input.xml
2024-12-04 14:17:27 [AWT-EventQueue-0] INFO Main:129 - Пользователь нажал кнопку 'Редактировать'.
2024-12-04 14:17:27 [AWT-EventQueue-0] WARN Main:138 - Попытка редактирования при выборе нескольких строк. Выбрано строк: 5
2024-12-04 14:17:29 [AWT-EventQueue-0] INFO Main:155 - Пользователь нажал кнопку 'Удалить'.
2024-12-04 14:17:30 [AWT-EventQueue-0] INFO Main:168 - Пользователь подтвердил удаление выделенных строк.
2024-12-04 14:17:30 [AWT-EventQueue-0] INFO Main:174 - Удаление выделенных строк завершено.
2024-12-04 14:17:33 [AWT-EventQueue-0] INFO Main:129 - Пользователь нажал кнопку 'Редактировать'.
2024-12-04 14:17:33 [AWT-EventQueue-0] INFO Main:145 - Открытие диалогового окна для редактирования строки с индексом: 1
2024-12-04 14:17:37 [AWT-EventQueue-0] INFO RecordDialog:115 - Начата проверка данных формы редактирования.
2024-12-04 14:17:37 [AWT-EventQueue-0] WARN RecordDialog:149 - Дата нарушения '20.07.2025' больше текущей даты.
2024-12-04 14:17:42 [AWT-EventQueue-0] INFO RecordDialog:115 - Начата проверка данных формы редактирования.
2024-12-04 14:17:42 [AWT-EventQueue-0] INFO RecordDialog:165 - Обновление записи в строке 1: Петров Петр Петрович, В456МН77,
2024-12-04 14:17:42 [AWT-EventQueue-0] INFO RecordDialog:172 - Сохранение данных успешно завершено.
2024-12-04 14:17:42 [AWT-EventQueue-0] INFO Main:147 - Диалоговое окно для редактирования закрыто.
2024-12-04 14:17:55 [AWT-EventQueue-0] INFO Main:186 - Пользователь нажал кнопку 'Поиск'.
2024-12-04 14:17:55 [AWT-EventQueue-0] INFO Main:264 - Начата проверка поля поиска.
2024-12-04 14:17:55 [AWT-EventQueue-0] INFO Main:190 - Проверка поля поиска успешно завершена. Поисковый запрос: '20'.
2024-12-04 14:17:55 [AWT-EventQueue-0] INFO Main:284 - Начат поиск. Запрос: '20'.
2024-12-04 14:17:55 [AWT-EventQueue-0] INFO Main:318 - Поиск завершен. Найдены совпадения.
2024-12-04 14:17:55 [AWT-EventQueue-0] INFO Main:193 - Поиск завершен.
2024-12-04 14:18:02 [AWT-EventQueue-0] INFO Main:215 - Пользователь нажал кнопку 'Сохранить'.
2024-12-04 14:18:02 [AWT-EventQueue-0] INFO Main:219 - Поток для сохранения данных запущен.
2024-12-04 14:18:02 [Thread-1] INFO Main:444 - Начато сохранение данных в XML файл.
2024-12-04 14:18:08 [Thread-1] INFO Main:453 - Выбран файл для сохранения: D:\Projects\00P_ETU\out
2024-12-04 14:18:08 [Thread-1] INFO Main:491 - Данные успешно сохранены в файл: D:\Projects\00P_ETU\out.xml
2024-12-04 14:18:11 [AWT-EventQueue-0] INFO Main:224 - Пользователь нажал кнопку 'Сформировать отчёт'.
2024-12-04 14:18:11 [AWT-EventQueue-0] INFO Main:228 - Поток для генерации HTML-отчета запущен.
2024-12-04 14:18:11 [Thread-2] INFO Main:510 - Начата генерация HTML-отчета.
2024-12-04 14:18:12 [Thread-2] INFO Main:538 - HTML-отчет успешно создан: report.html

```

Рисунок 2 – Логи в режиме *WARN+INFO*

```

2024-12-04 14:29:30 [main] INFO Main:553 - Запуск приложения GAI System.
2024-12-04 14:29:31 [main] INFO Main:557 - Приложение успешно запущено.
2024-12-04 14:29:32 [AWT-EventQueue-0] INFO Main:206 - Пользователь нажал кнопку 'Загрузить'.
2024-12-04 14:29:32 [AWT-EventQueue-0] DEBUG Main:208 - Создан поток для загрузки данных.
2024-12-04 14:29:32 [AWT-EventQueue-0] INFO Main:210 - Поток для загрузки данных запущен.
2024-12-04 14:29:32 [Thread-0] INFO Main:389 - Начата загрузка данных из XML файла.
2024-12-04 14:29:38 [Thread-0] INFO Main:398 - Выбран файл для загрузки: D:\Projects\00P_ETU\input.xml
2024-12-04 14:29:38 [Thread-0] DEBUG Main:401 - Инициализация XML-документа.
2024-12-04 14:29:38 [Thread-0] DEBUG Main:408 - Текущая таблица очищена.
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Иванов Иван Иванович, A123BC77, 15.03.2024, Превышение скорости
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Петров Петр Петрович, B456MN77, 20.07.2023, Проезд на красный свет
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Смирнова Анна Сергеевна, C789OP77, 05.05.2024, Нарушение парковки
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Кузнецова Мария Александровна, D123EK77, 12.12.2023, Отсутствие страховки
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Соколов Сергей Викторович, E456TP77, 22.02.2024, Разворот в неположенном месте
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Григорьев Алексей Николаевич, M678OP77, 10.08.2023, Неправильная парковка
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Федорова Марина Ивановна, H432KC77, 03.10.2024, Превышение скорости
2024-12-04 14:29:38 [Thread-0] DEBUG Main:422 - Добавлена запись: Ковалев Роман Алексеевич, P543CM77, 14.02.2023, Неоплаченный штраф
2024-12-04 14:29:39 [Thread-0] INFO Main:426 - Данные успешно загружены из файла: D:\Projects\00P_ETU\input.xml
2024-12-04 14:29:45 [AWT-EventQueue-0] INFO Main:155 - Пользователь нажал кнопку 'Удалить'.
2024-12-04 14:29:45 [AWT-EventQueue-0] DEBUG Main:164 - Количество выделенных строк для удаления: 7
2024-12-04 14:29:46 [AWT-EventQueue-0] INFO Main:168 - Пользователь подтвердил удаление выделенных строк.
2024-12-04 14:29:46 [AWT-EventQueue-0] DEBUG Main:171 - Удаление строки с индексом: 6
2024-12-04 14:29:46 [AWT-EventQueue-0] DEBUG Main:171 - Удаление строки с индексом: 5
2024-12-04 14:29:46 [AWT-EventQueue-0] DEBUG Main:171 - Удаление строки с индексом: 4
2024-12-04 14:29:46 [AWT-EventQueue-0] DEBUG Main:171 - Удаление строки с индексом: 3
2024-12-04 14:29:46 [AWT-EventQueue-0] DEBUG Main:171 - Удаление строки с индексом: 2
2024-12-04 14:29:46 [AWT-EventQueue-0] DEBUG Main:171 - Удаление строки с индексом: 1
2024-12-04 14:29:46 [AWT-EventQueue-0] DEBUG Main:171 - Удаление строки с индексом: 0
2024-12-04 14:29:46 [AWT-EventQueue-0] INFO Main:174 - Удаление выделенных строк завершено.
2024-12-04 14:29:51 [AWT-EventQueue-0] INFO Main:215 - Пользователь нажал кнопку 'Сохранить'.
2024-12-04 14:29:51 [AWT-EventQueue-0] DEBUG Main:217 - Создан поток для сохранения данных.
2024-12-04 14:29:51 [AWT-EventQueue-0] INFO Main:219 - Поток для сохранения данных запущен.
2024-12-04 14:29:51 [Thread-1] INFO Main:444 - Начато сохранение данных в XML файл.
2024-12-04 14:29:56 [Thread-1] INFO Main:453 - Выбран файл для сохранения: D:\Projects\00P_ETU\out
2024-12-04 14:29:56 [Thread-1] DEBUG Main:456 - К имени файла добавлено расширение .xml: D:\Projects\00P_ETU\out.xml
2024-12-04 14:29:56 [Thread-1] DEBUG Main:461 - Инициализация XML-документа для сохранения данных.
2024-12-04 14:29:56 [Thread-1] DEBUG Main:478 - Добавлена запись в XML: Ковалев Роман Алексеевич, P543CM77, 14.02.2023, Неоплаченный штраф
2024-12-04 14:29:56 [Thread-1] INFO Main:491 - Данные успешно сохранены в файл: D:\Projects\00P_ETU\out.xml
2024-12-04 14:29:58 [AWT-EventQueue-0] INFO Main:224 - Пользователь нажал кнопку 'Сформировать отчет'.
2024-12-04 14:29:58 [AWT-EventQueue-0] DEBUG Main:226 - Создан поток для генерации HTML-отчета.
2024-12-04 14:29:58 [AWT-EventQueue-0] INFO Main:228 - Поток для генерации HTML-отчета запущен.
2024-12-04 14:29:58 [Thread-2] INFO Main:510 - Начата генерация HTML-отчета.
2024-12-04 14:29:58 [Thread-2] DEBUG Main:515 - Компиляция шаблона отчета.
2024-12-04 14:29:59 [Thread-2] DEBUG Main:518 - Подготовка данных для отчета из модели таблицы.
2024-12-04 14:29:59 [Thread-2] DEBUG Main:525 - Установлены параметры для отчета: {ReportTitle=Отчет о данных ГАИ, Author=GAI System}
2024-12-04 14:29:59 [Thread-2] DEBUG Main:527 - Заполнение отчета с использованием данных и параметров.
2024-12-04 14:29:59 [Thread-2] DEBUG Main:532 - Генерация HTML-отчета в файл: report.html
2024-12-04 14:29:59 [Thread-2] INFO Main:538 - HTML-отчет успешно создан: report.html

```

Рисунок 3 - Логи в режиме *DEBUG*

Текст программы

Класс Main.java

```

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import javax.swing.filechooser.FileNameExtensionFilter;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.util.HashMap;

import org.w3c.dom.*;
import org.xml.sax.SAXException;

import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.*;
import javax.xml.transform.stream.*;

import net.sf.jasperreports.engine.*;
import net.sf.jasperreports.engine.data.JRTableModelDataSource;
import net.sf.jasperreports.export.HtmlExporter;
import net.sf.jasperreports.export.SimpleExporterInput;
import net.sf.jasperreports.export.SimpleHtmlExporterOutput;

import org.apache.logging.log4j.LogManager;

```

```

import org.apache.logging.log4j.Logger;

import Exceptions.*;

/**
 * Программа для работы с данными о водителях и их нарушениях.
 * Содержит функции добавления, редактирования, удаления записей, а также сохранения и
 * загрузки данных в файл.
 *
 * @author Шарапов Иван 3312
 * @version 1.8
 */
public class Main {
    private JFrame mainFrame;
    private DefaultTableModel tableModel;
    private JTable dataTable;
    private JTextField searchField;
    private JComboBox<String> searchTypeComboBox;

    private final Object syncObject = new Object();
    private boolean isDataLoaded = false;
    private static final Logger logger = LogManager.getLogger(Main.class);

    /**
     * Конструктор класса Main.
     * Инициализирует основное окно приложения для работы с данными.
     */
    public Main() {
    }

    /**
     * Метод для создания и отображения главного окна программы.
     * Включает создание таблицы, панели инструментов с кнопками и панель поиска.
     */
    public void show() {
        // Создание основного окна приложения
        mainFrame = new JFrame("GAI System");
        mainFrame.setSize(800, 400);
        mainFrame.setLocation(100, 100);
        mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Создание кнопок для управления записями
        JButton addDriverButton = new JButton("Добавить");
        JButton editDriverButton = new JButton("Редактировать");
        JButton deleteDriverButton = new JButton("Удалить");
        JButton loadDriverButton = new JButton("Загрузить");
        JButton saveDriverButton = new JButton("Сохранить");
        JButton generateReportButton = new JButton("Сформировать отчёт");

        // Панель инструментов, которая содержит кнопки
        JToolBar toolBar = new JToolBar("Toolbar");
        toolBar.setLayout(new BorderLayout()); // Устанавливаем BorderLayout для панели
        инструментов

        // Панель с кнопками управления (добавление, редактирование, удаление)
        JPanel leftPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        leftPanel.add(addDriverButton);
        leftPanel.add(editDriverButton);
        leftPanel.add(deleteDriverButton);
        toolBar.add(leftPanel, BorderLayout.WEST); // Размещаем в левой части панели

        // Панель с кнопками сохранения и загрузки
        JPanel rightPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        rightPanel.add(loadDriverButton);
        rightPanel.add(saveDriverButton);
        rightPanel.add(generateReportButton);
        toolBar.add(rightPanel, BorderLayout.EAST); // Размещаем в правой части панели

        mainFrame.setLayout(new BorderLayout());
        mainFrame.add(toolBar, BorderLayout.NORTH); // Размещение панель инструментов
        сверху

        // Создание таблицы для отображения данных
        String[] columns = {"ФИО водителя", "Номер машины", "Дата нарушения", "Тип
        нарушения"};
        String[][] data = {};
        tableModel = new DefaultTableModel(data, columns);
        dataTable = new JTable(tableModel); // Таблица, которая использует данные

```

```

tableModel
    JScrollPane scrollPane = new JScrollPane(dataTable); // Добавляем прокрутку для
таблицы
    mainFrame.add(scrollPane, BorderLayout.CENTER); // Размещаем таблицу в центре окна

    // Элементы поиска: поле ввода и кнопка "Поиск"
    searchTypeComboBox = new JComboBox<>(new String[]{"По имени", "По номеру машины",
"По дате нарушения", "По типу нарушения"});
    searchField = new JTextField(15);
    JButton searchButton = new JButton("Поиск");

    JPanel searchPanel = new JPanel();
    searchPanel.add(searchTypeComboBox);
    searchPanel.add(searchField);
    searchPanel.add(searchButton);
    mainFrame.add(searchPanel, BorderLayout.SOUTH); // Панель поиска размещается внизу

    // Добавляем действия для кнопок

    // "Добавить" — выполняет добавление новой записи
    addDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            logger.info("Пользователь нажал кнопку 'Добавить'.");
            logger.debug("Открытие диалогового окна для добавления новой записи.");
            new RecordDialog(mainFrame, tableModel, -1).setVisible(true); // -1
означает, что это добавление новой записи
            logger.info("Диалоговое окно для добавления записи закрыто.");
        }
    });

    // "Редактировать" — выполняет редактирование выбранной записи
    editDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            logger.info("Пользователь нажал кнопку 'Редактировать'.");
            int[] selectedRows = dataTable.getSelectedRows();

            // Проверка, что выбрана ровно одна строка
            if (selectedRows.length != 1) {
                if (selectedRows.length == 0) {
                    logger.warn("Попытка редактирования без выбора строки.");
                    JOptionPane.showMessageDialog(mainFrame, "Пожалуйста, выберите
строку для редактирования.", "Ошибка", JOptionPane.WARNING_MESSAGE);
                } else {
                    logger.warn("Попытка редактирования при выборе нескольких строк.
Выбрано строк: {}", selectedRows.length);
                    JOptionPane.showMessageDialog(mainFrame, "Пожалуйста, выберите
только одну строку для редактирования.", "Ошибка", JOptionPane.WARNING_MESSAGE);
                }
                return;
            }

            int selectedRow = selectedRows[0];
            logger.info("Открытие диалогового окна для редактирования строки с
индексом: {}", selectedRow);
            new RecordDialog(mainFrame, tableModel, selectedRow).setVisible(true);
            logger.info("Диалоговое окно для редактирования закрыто.");
        }
    });

    // "Удалить" — выполняет удаление выбранных записей
    deleteDriverButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            logger.info("Пользователь нажал кнопку 'Удалить'.");
            // Проверяем, есть ли выделенные строки
            int[] selectedRows = dataTable.getSelectedRows();
            if (selectedRows.length == 0) {
                logger.warn("Попытка удаления без выделения строк.");
                JOptionPane.showMessageDialog(mainFrame, "Нет выделенных строк для
удаления.", "Ошибка", JOptionPane.WARNING_MESSAGE);
                return;
            }

            logger.debug("Количество выделенных строк для удаления: {}",
selectedRows.length);
            // Запрос подтверждения у пользователя

```



```

        int confirm = JOptionPane.showConfirmDialog(mainFrame, "Вы уверены, что
        хотите удалить выделенные строки?", "Подтверждение удаления", JOptionPane.YES_NO_OPTION);
        if (confirm == JOptionPane.YES_OPTION) {
            logger.info("Пользователь подтвердил удаление выделенных строк.");
            // Удаляем строки с конца списка, чтобы избежать смещения индексов
            for (int i = selectedRows.length - 1; i >= 0; i--) {
                logger.debug("Удаление строки с индексом: {}", selectedRows[i]);
                tableModel.removeRow(selectedRows[i]);
            }
            logger.info("Удаление выделенных строк завершено.");
            JOptionPane.showMessageDialog(mainFrame, "Выделенные строки успешно
            удалены.", "Удаление", JOptionPane.INFORMATION_MESSAGE);
        } else {
            logger.info("Пользователь отменил удаление выделенных строк.");
        }
    }
});

// "Поиск" — выполняет поиск в таблице, по введённой строке
searchButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        logger.info("Пользователь нажал кнопку 'Поиск'.");
        try {
            logger.debug("Начата проверка поля поиска.");
            validateSearchField(searchField);
            logger.info("Проверка поля поиска успешно завершена. Поисковый запрос:
            '{}'.", searchField.getText());
            logger.debug("Начато выполнение поиска.");
            performSearch(searchField.getText());
            logger.info("Поиск завершен.");
        } catch (NullPointerException ex) {
            logger.error("Ошибка: Поисковый запрос отсутствует (null).", ex);
            JOptionPane.showMessageDialog(mainFrame, ex.getMessage(), "Ошибка",
            JOptionPane.ERROR_MESSAGE);
        } catch (EmptySearchException ex) {
            logger.warn("Ошибка: Поле поиска пустое.");
            JOptionPane.showMessageDialog(mainFrame, ex.getMessage(), "Ошибка",
            JOptionPane.WARNING_MESSAGE);
        }
    }
});

// "Загрузить" — открывает диалоговое окно для выбора файла и загружает данные в
таблицу
loadDriverButton.addActionListener(e -> {
    logger.info("Пользователь нажал кнопку 'Загрузить'.");
    Thread loadDataThread = createLoadDataThread();
    logger.debug("Создан поток для загрузки данных.");
    loadDataThread.start();
    logger.info("Поток для загрузки данных запущен.");
});

// "Сохранить" — открывает диалоговое окно для сохранения файла и записывает данные
таблицы в файл
saveDriverButton.addActionListener(e -> {
    logger.info("Пользователь нажал кнопку 'Сохранить'.");
    Thread saveDataThread = createSaveDataThread();
    logger.debug("Создан поток для сохранения данных.");
    saveDataThread.start();
    logger.info("Поток для сохранения данных запущен.");
});

// "Сформировать отчёт" — генерирует отчёт в формате HTML
generateReportButton.addActionListener(e -> {
    logger.info("Пользователь нажал кнопку 'Сформировать отчёт'.");
    Thread generateReportThread = createGenerateReportThread();
    logger.debug("Создан поток для генерации HTML-отчета.");
    generateReportThread.start();
    logger.info("Поток для генерации HTML-отчета запущен.");
});

// Делаем главное окно видимым
mainFrame.setVisible(true);
}

/**
 * Возвращает индекс столбца для поиска на основе выбранного поля.

```



```

*
* @param selectedField Поле, выбранное пользователем в JComboBox для поиска
* @return Индекс столбца для поиска, или -1, если поле не распознано
*/
private int getColumnIndex(String selectedField) {
    switch (selectedField) {
        case "По имени":
            return 0; // Индекс столбца с ФИО водителя
        case "По номеру машины":
            return 1; // Индекс столбца с номером машины
        case "По дате нарушения":
            return 2; // Индекс столбца с датой нарушения
        case "По типу нарушения":
            return 3; // Индекс столбца с типом нарушения
        default:
            return -1;
    }
}

/**
 * Проверяет значение в поле поиска и генерирует исключения, если поле пустое или
 * содержит null.
 */
* @param searchField Поле ввода текста для поиска
* @throws EmptySearchException Если поле ввода пустое
* @throws NullPointerException Если значение в поле null
*/
private void validateSearchField(JTextField searchField) throws EmptySearchException,
NullPointerException {
    logger.info("Начата проверка поля поиска.");
    String searchText = searchField.getText();
    if (searchText == null) {
        logger.error("Поисковый запрос отсутствует (null).");
        throw new NullPointerException("Поисковый запрос отсутствует");
    }
    if (searchText.isEmpty()) {
        logger.warn("Поисковый запрос пустой.");
        throw new EmptySearchException();
    }
    logger.debug("Проверка поля поиска завершена успешно. Поисковый запрос: '{}'.",
searchText);
}

/**
 * Выполняет поиск в таблице по указанному тексту.
 * Если совпадения найдены, строки выделяются.
 */
* @param query Строка для поиска.
*/
private void performSearch(String query) {
    logger.info("Начат поиск. Запрос: '{}'.", query);
    dataTable.clearSelection(); // Снимаем предыдущее выделение
    logger.debug("Снято предыдущее выделение строк таблицы.");
    boolean found = false;

    // Приводим запрос к нижнему регистру
    String lowerCaseQuery = query.toLowerCase();
    logger.debug("Запрос приведен к нижнему регистру: '{}'.", lowerCaseQuery);

    // Получаем индекс столбца для поиска
    int columnIndex = getColumnIndex((String) searchTypeComboBox.getSelectedItem());
    if (columnIndex == -1) {
        logger.error("Некорректное поле для поиска: '{}'.",
searchTypeComboBox.getSelectedItem());
        JOptionPane.showMessageDialog(mainFrame, "Некорректное поле для поиска",
"Ошибка", JOptionPane.ERROR_MESSAGE);
        return;
    }
    logger.debug("Поиск будет выполняться по столбцу с индексом: {}.", columnIndex);

    // Проходим по всем строкам, но только в выбранном столбце
    for (int i = 0; i < tableModel.getRowCount(); i++) {
        String cellValue = tableModel.getValueAt(i,
columnIndex).toString().toLowerCase();

        // Если значение ячейки содержит искомый текст без учета регистра
        if (cellValue.contains(lowerCaseQuery)) {
            dataTable.addRowSelectionInterval(i, i); // Выделяем строку

```

```

        logger.debug("Найдено совпадение в строке {}. ", i);
        found = true;
    }
}

if (!found) {
    logger.info("Поиск завершен. Совпадений не найдено.");
    JOptionPane.showMessageDialog(mainFrame, "Совпадения не найдены", "Результат
поиска", JOptionPane.INFORMATION_MESSAGE);
} else {
    logger.info("Поиск завершен. Найдены совпадения.");
}
}

/**
 * Создает поток для загрузки данных из XML-файла.
 * Поток синхронизирован и уведомляет другие потоки о завершении загрузки данных.
 *
 * @return Поток для загрузки данных.
 */
private Thread createLoadDataThread() {
    return new Thread(() -> {
        synchronized (syncObject) {
            loadData(); // Загрузка данных
            isDataLoaded = true; // Флаг завершения
            syncObject.notifyAll(); // Уведомляем другие потоки
        }
    });
}

/**
 * Создает поток для сохранения данных таблицы в XML-файл.
 * Поток ожидает завершения загрузки данных перед началом работы.
 *
 * @return Поток для сохранения данных.
 */
private Thread createSaveDataThread() {
    return new Thread(() -> {
        synchronized (syncObject) {
            try {
                while (!isDataLoaded) {
                    syncObject.wait(); // Ждем завершения загрузки
                }
                saveData(); // Сохранение данных
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });
}

/**
 * Создает поток для генерации HTML-отчета на основе данных таблицы.
 * Поток ожидает завершения загрузки данных перед началом работы.
 *
 * @return Поток для генерации отчета.
 */
private Thread createGenerateReportThread() {
    return new Thread(() -> {
        synchronized (syncObject) {
            try {
                while (!isDataLoaded) {
                    syncObject.wait(); // Ждем завершения загрузки
                    // *Поставленная в лабораторной работе задача ломает логику
                    // Ведь отчёт генерируется по данным из таблички, а не XML
                    // Потому что пользователь может сохранить XML куда угодно
                    // А может и не сохранить совсем
                }
                generateHtmlReport(); // Генерация отчёта
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });
}
}

/**

```

```

* Загружает данные из XML-файла и добавляет их в таблицу.
* Отображает диалоговое окно для выбора файла и обрабатывает ошибки загрузки.
*/
private void loadData() {
    logger.info("Начата загрузка данных из XML файла.");
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setCurrentDirectory(new File(System.getProperty("user.dir")));
    fileChooser.setDialogTitle("Открыть XML файл");
    fileChooser.setFileFilter(new FileNameExtensionFilter("XML файлы", "xml"));

    int userSelection = fileChooser.showOpenDialog(mainFrame);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        File fileToLoad = fileChooser.getSelectedFile();
        logger.info("Выбран файл для загрузки: {}", fileToLoad.getAbsolutePath());

        try {
            logger.debug("Инициализация XML-документа.");
            DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
            DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
            Document doc = dBuilder.parse(fileToLoad);
            doc.getDocumentElement().normalize();

            tableModel.setRowCount(0);
            logger.debug("Текущая таблица очищена.");

            NodeList driverNodes = doc.getElementsByTagName("driver");

            for (int i = 0; i < driverNodes.getLength(); i++) {
                Node node = driverNodes.item(i);
                NamedNodeMap attributes = node.getAttributes();

                String name = attributes.getNamedItem("name").getNodeValue();
                String license = attributes.getNamedItem("license").getNodeValue();
                String violationDate =
attributes.getNamedItem("violationDate").getNodeValue();
                String violationType =
attributes.getNamedItem("violationType").getNodeValue();

                tableModel.addRow(new Object[]{name, license, violationDate,
violationType});
                logger.debug("Добавлена запись: {}, {}, {}, {}", name, license,
violationDate, violationType);
            }

            JOptionPane.showMessageDialog(mainFrame, "Данные успешно загружены из XML
файла.");
            logger.info("Данные успешно загружены из файла: {}",
fileToLoad.getAbsolutePath());
        } catch (ParserConfigurationException | SAXException | IOException ex) {
            logger.error("Ошибка при загрузке данных из файла: {}",
fileToLoad.getAbsolutePath(), ex);
            JOptionPane.showMessageDialog(mainFrame, "Ошибка при загрузке данных из XML
файла.", "Ошибка", JOptionPane.ERROR_MESSAGE);
        }
    } else {
        logger.warn("Пользователь отменил выбор файла.");
    }
}

/**
* Сохраняет данные из таблицы в XML-файл.
* <p>
* Показывает диалоговое окно для выбора файла, затем записывает текущие
* данные из таблицы в указанный файл в формате XML. Обрабатывает возможные
* ошибки при сохранении файла.
*/
private void saveData() {
    logger.info("Начато сохранение данных в XML файл.");
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setCurrentDirectory(new File(System.getProperty("user.dir")));
    fileChooser.setDialogTitle("Сохранить как XML");
    fileChooser.setFileFilter(new FileNameExtensionFilter("XML файлы", "xml"));

    int userSelection = fileChooser.showSaveDialog(mainFrame);
    if (userSelection == JFileChooser.APPROVE_OPTION) {
        File fileToSave = fileChooser.getSelectedFile();
        logger.info("Выбран файл для сохранения: {}", fileToSave.getAbsolutePath());
        if (!fileToSave.getAbsolutePath().endsWith(".xml")) {

```

```

        fileToSave = new File(fileToSave + ".xml");
        logger.debug("К имени файла добавлено расширение .xml: {}",
fileToSave.getAbsolutePath());
    }

    try {
        // Создаем XML-документ
        logger.debug("Инициализация XML-документа для сохранения данных.");
        DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
        DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
        Document doc = dBuilder.newDocument();

        Element rootElement = doc.createElement("drivers");
        doc.appendChild(rootElement);

        for (int i = 0; i < tableModel.getRowCount(); i++) {
            Element driver = doc.createElement("driver");

            driver.setAttribute("name", (String) tableModel.getValueAt(i, 0));
            driver.setAttribute("license", (String) tableModel.getValueAt(i, 1));
            driver.setAttribute("violationDate", (String) tableModel.getValueAt(i,
2));
            driver.setAttribute("violationType", (String) tableModel.getValueAt(i,
3));

            rootElement.appendChild(driver);
            logger.debug("Добавлена запись в XML: {}, {}, {}, {}",
                tableModel.getValueAt(i, 0),
                tableModel.getValueAt(i, 1),
                tableModel.getValueAt(i, 2),
                tableModel.getValueAt(i, 3));
        }

        // Сохраняем XML-документ в файл
        TransformerFactory transformerFactory = TransformerFactory.newInstance();
        Transformer transformer = transformerFactory.newTransformer();
        DOMSource source = new DOMSource(doc);
        StreamResult result = new StreamResult(fileToSave);
        transformer.transform(source, result);
        logger.info("Данные успешно сохранены в файл: {}",
fileToSave.getAbsolutePath());
        JOptionPane.showMessageDialog(mainFrame, "Данные успешно сохранены в XML
файл.");
    } catch (ParserConfigurationException | TransformerException ex) {
        logger.error("Ошибка при сохранении данных в файл: {}",
fileToSave.getAbsolutePath(), ex);
        JOptionPane.showMessageDialog(mainFrame, "Ошибка при сохранении данных в
XML файл.", "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
    else {
        logger.warn("Пользователь отменил выбор файла для сохранения.");
    }
}

/**
 * Генерирует HTML-отчет на основе данных таблицы.
 * <p>
 * Создает отчет в формате HTML, используя текущие данные из таблицы.
 * Отчет сохраняется в файл, расположенный в текущей рабочей директории.
 * Обрабатывает возможные ошибки при создании отчета.
 * </p>
 */
private void generateHtmlReport() {
    logger.info("Начата генерация HTML-отчета.");
    try {
        // Путь к шаблону отчета
        String jrxmlPath = "src/main/resources/GAI.jrxml";

        logger.debug("Компиляция шаблона отчета.");
        JasperReport jasperReport = JasperCompileManager.compileReport(jrxmlPath);

        logger.debug("Подготовка данных для отчета из модели таблицы.");
        JRTableModelDataSource dataSource = new JRTableModelDataSource(tableModel);

        // Параметры для отчета (если нужны)
        HashMap<String, Object> parameters = new HashMap<>();
        parameters.put("ReportTitle", "Отчет о данных ГАИ");
        parameters.put("Author", "GAI System");
        logger.debug("Установлены параметры для отчета: {}", parameters);
    }
}

```

```

        logger.debug("Заполнение отчета с использованием данных и параметров.");
        JasperPrint jasperPrint = JasperFillManager.fillReport(jasperReport,
parameters, dataSource);

        // Генерация HTML-отчета
        String outputFilePath = "report.html";
        logger.debug("Генерация HTML-отчета в файл: {}", outputFilePath);
        HtmlExporter exporter = new HtmlExporter();
        exporter.setExporterInput(new SimpleExporterInput(jasperPrint));
        exporter.setExporterOutput(new SimpleHtmlExporterOutput(outputFilePath));

        exporter.exportReport();
        logger.info("HTML-отчет успешно создан: {}", outputFilePath);
        JOptionPane.showMessageDialog(mainFrame, "HTML-отчет успешно создан: " +
outputFilePath);
    } catch (Exception e) {
        logger.error("Ошибка при создании HTML-отчета.", e);
        JOptionPane.showMessageDialog(mainFrame, "Ошибка при создании отчета: " +
e.getMessage());
    }
}

/**
 * Точка входа в приложение. Запускает метод show() для отображения GUI.
 */
/**
 * @param args Аргументы командной строки (не используются).
 */
public static void main(String[] args) {
    Logger logger = LogManager.getLogger(Main.class);
    logger.info("Запуск приложения GAI System.");

    try {
        new Main().show();
        logger.info("Приложение успешно запущено.");
    } catch (Exception e) {
        logger.error("Критическая ошибка при запуске приложения.", e);
    }
}
}

```

Класс RecordDialog.java

```

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

/**
 * Диалоговое окно для добавления или редактирования записи в таблице.
 */
public class RecordDialog extends JDialog {

    /**
     * Поле для ввода ФИО водителя.
     */
    private JTextField nameField;

    /**
     * Поле для ввода номера машины.
     */
    private JTextField licenseField;

    /**
     * Поле для ввода даты нарушения.
     */
    private JTextField dateField;

    /**
     * Поле для ввода типа нарушения.
     */
    private JTextField violationField;

    /**
     * Модель таблицы, используемая для добавления или обновления записи.
     */
}

```

```

    */
    private DefaultTableModel tableModel;
    /**
     * Индекс строки для редактирования, -1 если добавляется новая запись.
     */
    private int rowIndex = -1;
    private static final Logger logger = LogManager.getLogger(RecordDialog.class);

    /**
     * Конструктор для создания окна записи (добавление или редактирование).
     *
     * @param parentFrame Родительское окно
     * @param tableModel Модель таблицы
     * @param rowIndex Индекс строки для редактирования (-1 для добавления новой записи)
     */
    public RecordDialog(JFrame parentFrame, DefaultTableModel tableModel, int rowIndex) {
        true;
        super(parentFrame, rowIndex == -1 ? "Добавить запись" : "Редактировать запись",
            this.tableModel = tableModel;
            this.rowIndex = rowIndex;

            setSize(350, 250);
            setLocationRelativeTo(parentFrame);
            setLayout(new BorderLayout(10, 10));

            // Панель с полями ввода
            JPanel inputPanel = new JPanel(new GridLayout(4, 2, 5, 5));
            nameField = new JTextField(15);
            licenseField = new JTextField(10);
            dateField = new JTextField(10);
            violationField = new JTextField(15);

            inputPanel.add(new JLabel("ФИО водителя:"));
            inputPanel.add(nameField);
            inputPanel.add(new JLabel("Номер машины:"));
            inputPanel.add(licenseField);
            inputPanel.add(new JLabel("Дата нарушения (ДД.ММ.ГГГГ):"));
            inputPanel.add(dateField);
            inputPanel.add(new JLabel("Тип нарушения:"));
            inputPanel.add(violationField);

            // Заполнение полей, если это редактирование
            if (rowIndex != -1) {
                nameField.setText((String) tableModel.getValueAt(rowIndex, 0));
                licenseField.setText((String) tableModel.getValueAt(rowIndex, 1));
                dateField.setText((String) tableModel.getValueAt(rowIndex, 2));
                violationField.setText((String) tableModel.getValueAt(rowIndex, 3));
            }

            // Панель с кнопками
            JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 10));
            JButton saveButton = new JButton(rowIndex == -1 ? "Сохранить" : "Обновить");
            JButton cancelButton = new JButton("Отмена");

            saveButton.addActionListener(new ActionListener() {
                @Override
                public void actionPerformed(ActionEvent e) {
                    if (validateAndSave()) {
                        dispose();
                    }
                }
            });

            cancelButton.addActionListener(e -> dispose());

            buttonPanel.add(saveButton);
            buttonPanel.add(cancelButton);

            // Добавление панелей в диалоговое окно
            add(inputPanel, BorderLayout.CENTER);
            add(buttonPanel, BorderLayout.SOUTH);
        }

        /**
         * Проверяет корректность данных и сохраняет их, если данные верны.
         *
         * @return true, если данные успешно сохранены, иначе false
         */
    }

```



```

*/
private boolean validateAndSave() {
    logger.info("Начата проверка данных формы редактирования.");
    if (nameField.getText().isEmpty() || licenseField.getText().isEmpty() ||
        dateField.getText().isEmpty() || violationField.getText().isEmpty()) {
        logger.warn("Проверка не пройдена: не все поля заполнены.");
        JOptionPane.showMessageDialog(this, "Все поля должны быть заполнены.",
            "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    }
    logger.debug("Все поля заполнены.");

    // Проверка формата российского номера
    if
(!licenseField.getText().matches("[АВЕКМНОРСТУХ]{1}\\\\d{3}[АВЕКМНОРСТУХ]{2}\\\\d{2,3}$")) {
        logger.warn("Проверка не пройдена: неверный формат номера '{}'.",
            licenseField.getText());
        JOptionPane.showMessageDialog(this, "Неверный формат номера. Введите российский
номер (например, А123ВС77).", "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    }
    logger.debug("Формат номера '{}' прошёл проверку.", licenseField.getText());

    // Проверка даты нарушения
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd.MM.yyyy");
    LocalDate violationDate;
    try {
        violationDate = LocalDate.parse(dateField.getText(), formatter);
    } catch (DateTimeParseException ex) {
        logger.error("Неверный формат даты '{}'.", dateField.getText(), ex);
        JOptionPane.showMessageDialog(this, "Неверный формат даты. Используйте
ДД.ММ.ГГГГ.", "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    }

    logger.debug("Дата '{}' успешно распознана.", dateField.getText());
    LocalDate currentDate = LocalDate.now();
    LocalDate fiveYearsAgo = currentDate.minusYears(5);

    // Проверка: дата не должна быть в будущем и не старше 5 лет
    if (violationDate.isAfter(currentDate)) {
        logger.warn("Дата нарушения '{}' больше текущей даты.", dateField.getText());
        JOptionPane.showMessageDialog(this, "Дата нарушения не может быть в будущем.",
            "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    } else if (violationDate.isBefore(fiveYearsAgo)) {
        logger.warn("Дата нарушения '{}' старше 5 лет.", dateField.getText());
        JOptionPane.showMessageDialog(this, "Нарушение не должно быть старше 5 лет.",
            "Ошибка", JOptionPane.WARNING_MESSAGE);
        return false;
    }
    logger.debug("Дата '{}' прошла проверку.", dateField.getText());

    // Сохранение данных
    if (rowIndex == -1) {
        logger.info("Добавление новой записи: {}, {}, {}, {}.",
            nameField.getText(), licenseField.getText(), dateField.getText(),
            violationField.getText());
        tableModel.addRow(new Object[]{nameField.getText(), licenseField.getText(),
            dateField.getText(), violationField.getText()});
    } else {
        logger.info("Обновление записи в строке {}: {}, {}, {}, {}.",
            rowIndex, nameField.getText(), licenseField.getText(),
            dateField.getText(), violationField.getText());
        tableModel.setValueAt(nameField.getText(), rowIndex, 0);
        tableModel.setValueAt(licenseField.getText(), rowIndex, 1);
        tableModel.setValueAt(dateField.getText(), rowIndex, 2);
        tableModel.setValueAt(violationField.getText(), rowIndex, 3);
    }
    logger.info("Сохранение данных успешно завершено.");
    return true;
}
}

```

Класс EmptySearchException.java

```
package Exceptions;
```



```
/**
 * Исключение, вызываемое, когда поле поиска пустое.
 */
public class EmptySearchException extends Exception {

    /**
     * Конструктор без параметров, создающий исключение с сообщением об ошибке пустого поля
     * поиска.
     */
    public EmptySearchException() {
        super("Поле поиска не должно быть пустым");
    }
}
```

Приложение

Ссылка на репозиторий:

https://github.com/DexTver/OOP_ETU/tree/lab_10